INFO 4490 Project 1: Exploratory Analysis

Zhalae Daneshvari, Nick Johnson, Jolene Le

Objective(s)

State the question(s) you are answering or the problem(s) you are solving clearly.

The goal of this project is to develop a **supervised binary classification model** that identifies whether an adult is *likely prediabetic* based solely on self-reported survey data from the NHANES Continuous Questionnaire (August 2021–August 2023). The model predicts the probability that each respondent falls within clinical prediabetes ranges for either **fasting plasma glucose** (LBXGLU: 100–125 mg/dL) or **glycohemoglobin (HbA1c)** (LBXGH: 5.7–6.4%).

Our specific objectives are to:

1. **Train a binary classifier** (e.g., logistic regression or random forest) that estimates the probability of prediabetes using selected demographic, lifestyle, and self-reported health features.
2. **Label individuals as "likely prediabetic" or "not likely prediabetic"** by applying a probability threshold, effectively identifying who should be referred for confirmatory laboratory testing.
3. **Evaluate model performance** by comparing predicted labels to true lab-based outcomes using metrics such as accuracy, recall, precision, and ROC AUC.
4. **Demonstrate the potential of survey-based screening** as a low-cost, scalable method to guide early detection and prioritize follow-up testing.

Why this matters:
Prediabetes is widespread yet often undiagnosed, as many adults do not undergo routine blood testing. A survey-based model that flags individuals as *likely prediabetic* provides a simple, accessible first step—allowing clinics and community health programs to focus lab resources on those most at risk, promote earlier diagnosis, and help prevent progression to Type 2 diabetes.

Data collection and cleaning

Have an initial draft of your data cleaning appendix. Document every step that takes your raw data file(s) and turns it into the analysis-ready data set that you would submit with your final project. Include text narrative describing your data collection (downloading, scraping, surveys, etc) and any additional data curation/cleaning (merging data frames, filtering, transformations of variables, etc). Include code for data curation/cleaning, but not collection.

Data was pulled from August 2021-August 2023 Questionnaire Data - Continuous NHANES that contains individual xpt documents for each of the following labratory tests, examinations, and questionaires.

Here are the specific features we decided to use:

- Respondent Number (Join Key, *found in every table*)
  - SEQN - Respondent sequence number
- Weight history
  - WHD010 - Current self-reported height (inches)
  - WHD020 - Current self-reported weight (pounds)
  - WHD050 - Self-reported weight - 1 yr ago (pounds)
  - WHQ070 - Tried to lose weight in past year
- Reproductive Health
  - RHQ031 Had regular periods in past 12 months
  - RHQ131 - Ever been pregnant?
- Sleep Disorders
  - SLD012 - Sleep hours - weekdays or workdays
- Cigarette Use
  - SMQ020 - Smoked at least 100 cigarettes in life SMQ040 - Do you now smoke cigarettes?
- Physical Activity
  - PAD680 - Minutes sedentary activity
- Mental Health - Depression Screener
  - DPQ050 - Poor appetite or overeating DPQ020 - Feeling down, depressed, or hopeless
- Medical Conditions
  - MCQ010 - Ever been told you have asthma
  - MCQ053 - Taking treatment for anemia/past 3 mos
  - MCQ160b - Ever told had congestive heart failure
  - MCQ160e - Ever told you had heart attack
  - MCQ160f - Ever told you had a stroke
  - MCQ160m - Ever told you had thyroid problem
  - MCQ160l - Ever told you had any liver condition
  - MCQ510a - Liver condition: Fatty liver
  - MCQ510b - Liver condition: Liver fibrosis
  - MCQ510c - Liver condition: Liver cirrhosis
  - MCQ510d - Liver condition: Viral hepatitis
  - MCQ510e - Liver condition: Autoimmune hepatitis
- Kidney Conditions - Urology
  - KIQ022 - Ever told you had weak/failing kidneys?
- Body Measures
  - BMXBMI - Body MAss Index (kg/m**2)
- Diabetes
  - DIQ010 - Doctor told you have diabetes (DROP participants that have diabetes)
- Blood Pressure and Cholesterol

- BPQ020 - Ever told you had high blood pressure
- BPQ080 - Doctor told you - high cholesterol level

- Alcohol Use
  - ALQ130 - Avg # alcoholic drinks/day/past 12 mos
- Demographics
  - RIAGENDR - Gender
  - RIDAGEYR - Age in years at screening
  - RIDRETH3 - Race/Hispanic origin w/ NH Asian
- Target Column - Plasma Fasting Glucose
  - LBXGLU - Fasting Glucose (mg/dL)
- Target Column - Glycohemoglobin
  - LBXGH - Glycohemoglobin (%)

Our collection process was as follows:

1) Read and Standardize: Load each xpt file, keeping SEQN as key to join files
2) Merge Fasting Glucose and Glycohemoglobin (GHB/GLU targets), Diabetes Questionaire (exclusions), and all other chosen survey questions and demographics.
3) Filter: Drop rows of child respondents. (RIDAGEYR < 18). Drop rows without Fasting Glucose and Glycohemoglobin (GHB/GLU) present. Drop rows of respondents who are diagnosed with diabetes as we want to predict if someone is prediabetic from the set of respondents who do not already have diabetes.
4) Rename variables to clear, readable names (e.g. PAD680: sedentary_minutes) Add labeled answer columns for many yes/no or categorical items (e.g. RIAGENDR=1.0: sex=male)
5) Enforce numeric datatypes for numeric columns and set values outside of plausible ranges to NA. Ranges are as follows: 1) "age_years": (18, 99), "bmi": (10, 80), "A1c": (3, 20),"FPG": (50, 500),"sleep_hours_weekdays": (0, 24),"sedentary_minutes": (0, 1440),"drinks_per_day_12mo": (0, 50),"height_in": (40, 84),"weight_lb": (60, 700),"weight_1y_ago_lb": (60, 700).
6) Compute Self-Reported BMI feature where BMI = (self reported weight (kg))/(height (m)^2)
7) Compute Change in weight feature where change in weight = current weight minus self-reported weight 1 year ago. This will be positive if respondent gained weight and negative if they've lost weight.
8) Map response to survey question: Have you ever smoked 100+ cigarettes? to: 1 if they responded "Yes", 0 if they responded "No" or Nan if respondent refused, answered "Don't know" or if response is missing.
9) Map response to survey question: Do you smoke now? to 1 if they responded "Every day" or "Some days", 0 if they responded "Not at all", and Nan if they responded "Refused", "Don't know", or did not respond.
10) Drop columns that wont be used in the model such as weight, cycle, and ever_pregnant columns.
11) Drop Fasting Glucose and Glycohemoglobin Columns derived from lab tests that were used to construct target prediabetes column.
12) Drop columns used to remove respondents who are already diabetic.

```python
import os, warnings
from pathlib import Path
import pandas as pd, numpy as np

# Settings
NHANES_DIR = "data-raw/nhanes_xpt"
OUT_CSV = "data-raw/nhanes2123_prediabetes_features_labeled.csv"
CYCLES = ["L"]  # 2021–2023
ADULTS_ONLY = True  # keep age >= 18
EXCLUDE_DIAGNOSED = True  # drop diagnosed DM / on meds
REQUIRE_LAB_FOR_ROW = True  # require A1c or FPG present
REQUIRE_FASTING_FOR_WEIGHT = False  # use WTSAF2YR if True & available


# Helpers
def xpt_path(prefix, L):
    p1 = os.path.join(NHANES_DIR, f"{prefix}_{L}.XPT")
    p2 = os.path.join(NHANES_DIR, f"{prefix}_{L}.xpt")
    return p1 if os.path.exists(p1) else (p2 if os.path.exists(p2) else None)


def read_xpt(prefix, L):
    p = xpt_path(prefix, L)
    if not p:
        warnings.warn(f"Missing {prefix}_{L}.XPT; skipping.")
        return pd.DataFrame()
    df = pd.read_sas(p, format="xport")
    df.columns = [c.upper() for c in df.columns]
    if "SEQN" not in df.columns:
        warnings.warn(f"{prefix}_{L}: no SEQN; skipping.")
        return pd.DataFrame()
    return df


def safe_merge(
    left: pd.DataFrame, right: pd.DataFrame, src: str, key="SEQN"
) -> pd.DataFrame:
    """Left-merge on SEQN. If a non-key column collides, prefix it with
SRC__."""
    if right.empty:
        return left
    r = right.copy()
    collisions = [c for c in r.columns if c != key and c in left.columns]
    if collisions:
        r.rename(columns={c: f"{src}__{c}" for c in collisions}, inplace=True)
    return left.merge(r, on=key, how="left", copy=False)
```

```python
# Label Dictionsaries
YN = {1: "Yes", 2: "No", 7: "Refused", 9: "Don't know"}
SMOKE_NOW = {
    1: "Every day",
    2: "Some days",
    3: "Not at all",
    7: "Refused",
    9: "Don't know",
}
SEX_L = {1: "Male", 2: "Female"}
RIDRETH3_L = {
    1: "Mexican American",
    2: "Other Hispanic",
    3: "Non-Hispanic White",
    4: "Non-Hispanic Black",
    6: "Non-Hispanic Asian",
    7: "Other/Multiracial",
}

# Rename map
RENAME = {
    # Targets
    "LBXGH": "a1c_percent",  # A1c (%)
    "LBXGLU": "fpg_mg_dl",  # fasting plasma glucose (mg/dL)
    # Demographics / weight
    "RIDAGEYR": "age_years",
    "RIAGENDR": "sex_code",
    "RIDRETH3": "race_eth_code",
    "WTMEC2YR": "weight_sample",  # MEC sample weight (default)
    "WTSAF2YR": "weight_fasting",  # fasting subsample weight (from GLU)
    # Weight history (WHQ)
    "WHD010": "height_in",
    "WHD020": "weight_lb",
    "WHD050": "weight_1y_ago_lb",
    "WHQ070": "tried_lose_weight_past_year_code",
    # Reproductive (RHQ)
    "RHQ031": "regular_periods_12mo_code",
    "RHQ131": "ever_pregnant_code",
    # Sleep (SLQ)
    "SLD012": "sleep_hours_weekdays",
    # Cigarette use (SMQ)
    "SMQ020": "smoked_100_cigs_code",
    "SMQ040": "smoke_now_code",
    # Physical activity (PAD)
    "PAD680": "sedentary_minutes",
    # Depression (DPQ)
    "DPQ050": "dpq_poor_appetite_code",
    "DPQ020": "dpq_feeling_down_code",
```

```
    # Medical conditions (MCQ)
    "MCQ010": "ever_asthma_code",
    "MCQ053": "anemia_treatment_3mo_code",
    "MCQ160B": "ever_chf_code",
    "MCQ160E": "ever_heart_attack_code",
    "MCQ160F": "ever_stroke_code",
    "MCQ160M": "ever_thyroid_problem_code",
    "MCQ160L": "ever_liver_condition_code",
    "MCQ510A": "liver_fatty_code",
    "MCQ510B": "liver_fibrosis_code",
    "MCQ510C": "liver_cirrhosis_code",
    "MCQ510D": "liver_viral_hep_code",
    "MCQ510E": "liver_autoimmune_hep_code",
    # Kidney (KIQ)
    "KIQ022": "ever_weak_kidneys_code",
    # Body measures (BMX)
    "BMXBMI": "bmi",
    # Diabetes (DIQ) — for exclusions
    "DIQ010": "diq_diag_diabetes_code",
    "DIQ050": "diq_now_insulin_code",
    "DIQ070": "diq_now_pills_code",
    # BP & Cholesterol (BPQ)
    "BPQ020": "ever_high_bp_code",
    "BPQ080": "ever_high_chol_code",
    # Alcohol (ALQ)
    "ALQ130": "drinks_per_day_12mo",
}


# Build Cycle
def build_cycle(L: str) -> pd.DataFrame:
    # Required demographics/weights
    demo = read_xpt("DEMO", L)
    if demo.empty:
        raise SystemExit(f"DEMO_{L} is required (age/sex/race/weights).")

    # Targets
    ghb = read_xpt("GHB", L)  # A1c
    glu = read_xpt("GLU", L)  # FPG (+ WTSAF2YR)

    # Exclusions
    diq = read_xpt("DIQ", L)  # diagnosed DM / meds

    # Requested features
    whq = read_xpt("WHQ", L)
    rhq = read_xpt("RHQ", L)
    slq = read_xpt("SLQ", L)
    smq = read_xpt("SMQ", L)
```

```
    pad = read_xpt("PAQ", L)
    dpq = read_xpt("DPQ", L)
    mcq = read_xpt("MCQ", L)
    kiq = read_xpt("KIQ_U", L)
    bmx = read_xpt("BMX", L)
    bpq = read_xpt("BPQ", L)
    alq = read_xpt("ALQ", L)

    out = demo[["SEQN"]].copy()

    # Merge targets
    if not ghb.empty and "LBXGH" in ghb.columns:
        out = safe_merge(out, ghb[["SEQN", "LBXGH"]], "GHB")
    if not glu.empty:
        keep = [c for c in ["SEQN", "LBXGLU", "WTSAF2YR"] if c in glu.columns]
        out = safe_merge(out, glu[keep], "GLU")

    # Merge exclusions (DIQ)
    if not diq.empty:
        keep = [c for c in ["SEQN", "DIQ010", "DIQ050", "DIQ070"] if c in
diq.columns]
        out = safe_merge(out, diq[keep], "DIQ")

    # Merge requested features (best-effort)
    if not whq.empty:
        out = safe_merge(
            out,
            whq[
                [
                    c
                    for c in ["SEQN", "WHD010", "WHD020", "WHD050", "WHQ070"]
                    if c in whq.columns
                ]
            ],
            "WHQ",
        )
    if not rhq.empty:
        out = safe_merge(
            out,
            rhq[[c for c in ["SEQN", "RHQ031", "RHQ131"] if c in
rhq.columns]],
            "RHQ",
        )
    if not slq.empty:
        out = safe_merge(
            out, slq[[c for c in ["SEQN", "SLD012"] if c in slq.columns]],
"SLQ"
        )
```

```
    if not smq.empty:
        out = safe_merge(
            out,
            smq[[c for c in ["SEQN", "SMQ020", "SMQ040"] if c in
smq.columns]],
            "SMQ",
        )
    if not pad.empty:
        out = safe_merge(
            out, pad[[c for c in ["SEQN", "PAD680"] if c in pad.columns]],
"PAD"
        )
    if not dpq.empty:
        out = safe_merge(
            out,
            dpq[[c for c in ["SEQN", "DPQ050", "DPQ020"] if c in
dpq.columns]],
            "DPQ",
        )
    if not mcq.empty:
        keep = [
            c
            for c in [
                "SEQN",
                "MCQ010",
                "MCQ053",
                "MCQ160B",
                "MCQ160E",
                "MCQ160F",
                "MCQ160M",
                "MCQ160L",
                "MCQ510A",
                "MCQ510B",
                "MCQ510C",
                "MCQ510D",
                "MCQ510E",
            ]
            if c in mcq.columns
        ]
        out = safe_merge(out, mcq[keep], "MCQ")
    if not kiq.empty:
        out = safe_merge(
            out, kiq[[c for c in ["SEQN", "KIQ022"] if c in kiq.columns]],
"KIQ"
        )
    if not bmx.empty:
        out = safe_merge(
            out, bmx[[c for c in ["SEQN", "BMXBMI"] if c in bmx.columns]],
```

```
"BMX"
        )
    if not bpq.empty:
        out = safe_merge(
            out,
            bpq[[c for c in ["SEQN", "BPQ020", "BPQ080"] if c in
bpq.columns]],
            "BPQ",
        )
    if not alq.empty:
        out = safe_merge(
            out, alq[[c for c in ["SEQN", "ALQ130"] if c in alq.columns]],
"ALQ"
        )

    # Add demographics we need
    demo_keep = [
        c
        for c in ["SEQN", "RIDAGEYR", "RIAGENDR", "RIDRETH3", "WTMEC2YR"]
        if c in demo.columns
    ]
    out = safe_merge(out, demo[demo_keep], "DEMO")

    # Adult filter
    if ADULTS_ONLY and "RIDAGEYR" in out.columns:
        out = out[pd.to_numeric(out["RIDAGEYR"], errors="coerce") >=
18].copy()

    # Require at least one lab?
    a1c = pd.to_numeric(out.get("LBXGH"), errors="coerce")
    fpg = pd.to_numeric(out.get("LBXGLU"), errors="coerce")
    if REQUIRE_LAB_FOR_ROW:
        out = out.loc[(a1c.notna() | fpg.notna())].copy()

    # Prediabetes flag (Target)
    out["prediabetes_flag"] = (
        a1c.between(5.7, 6.4, inclusive="both")
        | fpg.between(100, 125, inclusive="both")
    ).astype("Int64")

    # Exclude diagnosed DM / meds now
    if EXCLUDE_DIAGNOSED:
        d_dm = out.get("DIQ010")
        d_ins = out.get("DIQ050")
        d_pil = out.get("DIQ070")

        def yes(s):
            return (
```

```
                s.map({1: 1, 2: 0, 3: 0, 7: np.nan, 9: np.nan})
                if s is not None
                else pd.Series(index=out.index, dtype="float64")
            )

        mask = (yes(d_dm) == 1) | (yes(d_ins) == 1) | (yes(d_pil) == 1)
        out = out.loc[~mask.fillna(False)].copy()

    # Choose weight (no cycle scaling for single release)
    if REQUIRE_FASTING_FOR_WEIGHT and "WTSAF2YR" in out.columns:
        w = pd.to_numeric(out["WTSAF2YR"], errors="coerce")
    else:
        w = pd.to_numeric(out.get("WTMEC2YR"), errors="coerce")
    out["weight"] = w

    # Rename columns to readable names
    present_map = {k: v for k, v in RENAME.items() if k in out.columns}
    out.rename(columns=present_map, inplace=True)

    # Add readable answers
    if "sex_code" in out.columns:
        out["sex"] = out["sex_code"].map(SEX_L)
    if "race_eth_code" in out.columns:
        out["race_eth"] = out["race_eth_code"].map(RIDRETH3_L)

    yn_vars = [
        "tried_lose_weight_past_year_code",
        "regular_periods_12mo_code",
        "ever_pregnant_code",
        "smoked_100_cigs_code",
        "dpq_poor_appetite_code",
        "dpq_feeling_down_code",
        "ever_asthma_code",
        "anemia_treatment_3mo_code",
        "ever_chf_code",
        "ever_heart_attack_code",
        "ever_stroke_code",
        "ever_thyroid_problem_code",
        "ever_liver_condition_code",
        "liver_fatty_code",
        "liver_fibrosis_code",
        "liver_cirrhosis_code",
        "liver_viral_hep_code",
        "liver_autoimmune_hep_code",
        "ever_weak_kidneys_code",
        "ever_high_bp_code",
        "ever_high_chol_code",
        "diq_diag_diabetes_code",
```

```
    ]
    for c in yn_vars:
        if c in out.columns:
            out[c.replace("_code", "_ans")] = out[c].map(YN)

    if "smoke_now_code" in out.columns:
        out["smoke_now_ans"] = out["smoke_now_code"].map(SMOKE_NOW)

    # Sliases for targets
    if "a1c_percent" in out.columns:
        out["A1c"] = pd.to_numeric(out["a1c_percent"], errors="coerce")
    if "fpg_mg_dl" in out.columns:
        out["FPG"] = pd.to_numeric(out["fpg_mg_dl"], errors="coerce")

    # Final tidy
    out["SEQN"] = pd.to_numeric(out["SEQN"], errors="coerce").astype("Int64")
    out["cycle"] = L

    # Snap tiny float dust to 0
    eps = 1e-12
    numcols = out.select_dtypes(include=["float", "int"]).columns
    out[numcols] = out[numcols].apply(pd.to_numeric, errors="coerce")
    out[numcols] = out[numcols].mask(out[numcols].abs() < eps, 0.0)

    # Column order
    order = [
        c
        for c in [
            "prediabetes_flag",
            "SEQN",
            "cycle",
            "weight",
            "age_years",
            "sex_code",
            "sex",
            "race_eth_code",
            "race_eth",
            "height_in",
            "weight_lb",
            "weight_1y_ago_lb",
            "bmi",
            "sleep_hours_weekdays",
            "sedentary_minutes",
            "drinks_per_day_12mo",
            "smoked_100_cigs_code",
            "smoked_100_cigs_ans",
            "smoke_now_code",
            "smoke_now_ans",
```

```
            "tried_lose_weight_past_year_code",
            "tried_lose_weight_past_year_ans",
            "regular_periods_12mo_code",
            "regular_periods_12mo_ans",
            "ever_pregnant_code",
            "ever_pregnant_ans",
            "dpq_poor_appetite_code",
            "dpq_poor_appetite_ans",
            "dpq_feeling_down_code",
            "dpq_feeling_down_ans",
            "ever_asthma_code",
            "ever_asthma_ans",
            "anemia_treatment_3mo_code",
            "anemia_treatment_3mo_ans",
            "ever_chf_code",
            "ever_chf_ans",
            "ever_heart_attack_code",
            "ever_heart_attack_ans",
            "ever_stroke_code",
            "ever_stroke_ans",
            "ever_thyroid_problem_code",
            "ever_thyroid_problem_ans",
            "ever_liver_condition_code",
            "ever_liver_condition_ans",
            "liver_fatty_code",
            "liver_fatty_ans",
            "liver_fibrosis_code",
            "liver_fibrosis_ans",
            "liver_cirrhosis_code",
            "liver_cirrhosis_ans",
            "liver_viral_hep_code",
            "liver_viral_hep_ans",
            "liver_autoimmune_hep_code",
            "liver_autoimmune_hep_ans",
            "ever_weak_kidneys_code",
            "ever_weak_kidneys_ans",
            "ever_high_bp_code",
            "ever_high_bp_ans",
            "ever_high_chol_code",
            "ever_high_chol_ans",
            "A1c",
            "FPG",
            "a1c_percent",
            "fpg_mg_dl",
            "weight_sample",
            "weight_fasting",
        ]
        if c in out.columns
```

```
    ]
    # append any remaining columns not listed
    order += [c for c in out.columns if c not in order]
    out = out[order]

    return out
```

```
Path(os.path.dirname(OUT_CSV) or ".").mkdir(parents=True, exist_ok=True)
parts = [build_cycle(L) for L in CYCLES]
df = pd.concat(parts, ignore_index=True)

# Deduplicate any duplicate 'weight' columns and drop invalid weights
if (df.columns == "weight").sum() > 1:
    warnings.warn(f"Duplicate 'weight' columns detected ({(df.columns ==
'weight').sum()}). Combining.")
    wcols = df.loc[:, df.columns == "weight"]
    w = wcols.bfill(axis=1).iloc[:, 0]
    df = df.drop(columns=[c for c in wcols.columns if c != wcols.columns[0]])
    df = df.rename(columns={wcols.columns[0]: "weight"})
    df["weight"] = pd.to_numeric(w, errors="coerce")
elif "weight" in df.columns:
    df["weight"] = pd.to_numeric(df["weight"], errors="coerce")

if "weight" in df.columns:
    before = len(df)
    df = df[df["weight"] > 0].copy()
    print(f"Dropped rows with invalid/nonpositive weight: {before - len(df)}")

df.to_csv(OUT_CSV, index=False)
print(f"Saved {OUT_CSV}  rows={len(df)}  cols={df.shape[1]}")
```

```
Dropped rows with invalid/nonpositive weight: 0
Saved data-raw/nhanes2123_prediabetes_features_labeled.csv  rows=5099  cols=70
```

```
import pandas as pd, numpy as np, os, json, math

IN_CSV = "data-raw/nhanes2123_prediabetes_features_labeled.csv"
OUT_CSV = "data-raw/nhanes2123_prediabetes_features_labeled_clean.csv"
OUT_DD = "data-raw/nhanes2123_prediabetes_features_datadict.json"

df = pd.read_csv(IN_CSV)

# Make sure SEQN is integer & unique
if "SEQN" in df.columns:
    df["SEQN"] = pd.to_numeric(df["SEQN"], errors="coerce").astype("Int64")
```

```
    if df["SEQN"].duplicated().any():
        df["_lab_count"] = (
            df[["A1c", "FPG"]].apply(lambda r: r.notna().sum(), axis=1)
            if {"A1c", "FPG"} <= set(df.columns)
            else 0
        )
        df = (
            df.sort_values(["SEQN", "_lab_count"], ascending=[True, False])
            .drop_duplicates(subset=["SEQN"], keep="first")
            .drop(columns=["_lab_count"])
        )
else:
    raise SystemExit("SEQN column is required.")

# Collapse duplicate 'weight' columns; drop nonpositive weights
wcols = [c for c in df.columns if c == "weight"]
if len(wcols) > 1:
    w = df[wcols].bfill(axis=1).iloc[:, 0]
    df = df.drop(columns=wcols).assign(weight=w)
elif len(wcols) == 1:
    df["weight"] = pd.to_numeric(df["weight"], errors="coerce")
else:
    # try to pick one from available sample weights
    for cand in ["weight_sample", "weight_fasting", "WTMEC2YR", "WTSAF2YR"]:
        if cand in df.columns:
            df["weight"] = pd.to_numeric(df[cand], errors="coerce")
            break

if "weight" in df.columns:
    df = df[df["weight"].fillna(0) > 0].copy()

# Coerce numerics & snap tiny float dust to 0
num_like = [
    "age_years",
    "bmi",
    "A1c",
    "FPG",
    "a1c_percent",
    "fpg_mg_dl",
    "sleep_hours_weekdays",
    "sedentary_minutes",
    "drinks_per_day_12mo",
    "height_in",
    "weight_lb",
    "weight_1y_ago_lb",
]
for c in num_like:
    if c in df.columns:
```

```
        df[c] = pd.to_numeric(df[c], errors="coerce")

eps = 1e-12
for c in df.select_dtypes(include=["float", "int"]).columns:
    df[c] = pd.to_numeric(df[c], errors="coerce")
    df[c] = df[c].mask(df[c].abs() < eps, 0.0)

# Range sanity (set out-of-bounds to NA)
ranges = {
    "age_years": (18, 99),
    "bmi": (10, 80),
    "A1c": (3, 20),
    "FPG": (50, 500),
    "sleep_hours_weekdays": (0, 24),
    "sedentary_minutes": (0, 1440),
    "drinks_per_day_12mo": (0, 50),
    "height_in": (40, 84),
    "weight_lb": (60, 700),
    "weight_1y_ago_lb": (60, 700),
}
for c, (lo, hi) in ranges.items():
    if c in df.columns:
        s = pd.to_numeric(df[c], errors="coerce")
        df.loc[(s < lo) | (s > hi), c] = np.nan

# Engineer a few helper features
# Computes BMI = weight(kg) / height(m)^2.
if {"height_in", "weight_lb"} <= set(df.columns):
    h_m = df["height_in"] * 0.0254
    w_kg = df["weight_lb"] * 0.45359237
    with np.errstate(invalid="ignore", divide="ignore"):
        df["bmi_from_selfreport"] = (w_kg / (h_m**2)).where((h_m > 0) & (w_kg
> 0))

# Change in weight: current weight minus self-reported weight 1 year ago
# Positive: gained weight; Negative: lost weight
if {"weight_lb", "weight_1y_ago_lb"} <= set(df.columns):
    df["weight_change_1y_lb"] = df["weight_lb"] - df["weight_1y_ago_lb"]

# Maps Nhanes: Ever smoked 100+ cigarettes? to :
# Code 1(smoker): 1
# Code 2(never smoker): 0
# Code 7 (Refused), Code 9 (Dont know), missing: Nan
if "smoked_100_cigs_code" in df.columns:
    df["ever_smoker"] = df["smoked_100_cigs_code"].map({1: 1, 2:
0}).astype("float")

# Maps Nhanes: Do you smoke now? to:
```

```
# Code 1 (Every day) : 1
# Code 2 (Some days) : 1
# Code 3 (Not at all) : 0
# Code 7 (Refused), Code 9 (Dont know), Missing : Nan
if "smoke_now_code" in df.columns:
    df["current_smoker"] = df["smoke_now_code"].map({1: 1, 2: 1, 3:
0}).astype("float")

# Drop weight columns, cycle, Nan columns, and sex/race code columns, and
columns used to construct prediabetes target or remove respondents with
diabetes
df = df.drop(
    columns=[
        "cycle",
        "weight",
        "weight_sample",
        "weight_fasting",
        "sex_code",
        "race_eth_code",
        "ever_pregnant_code",
        "ever_pregnant_ans",
        "A1c",
        "FPG",
        "a1c_percent",
        "fpg_mg_dl",
        "diq_diag_diabetes_code",
        "diq_now_insulin_code",
        "diq_now_pills_code",
        "diq_diag_diabetes_ans",
    ],
    errors="ignore",
)

# Write cleaned CSV
os.makedirs(os.path.dirname(OUT_CSV), exist_ok=True)
df.to_csv(OUT_CSV, index=False)

#  Data dictionary
desc = {
    "SEQN": "Respondent sequence number (unique within release)",
    "weight": "Analysis weight (MEC default; fasting weight if chosen)",
    "A1c": "Glycohemoglobin (%)",
    "FPG": "Fasting plasma glucose (mg/dL)",
    "prediabetes_flag": "1 if A1c 5.7–6.4% or FPG 100–125 mg/dL",
    "age_years": "Age in years at screening",
    "sex_code / sex": "1=Male, 2=Female / labeled",
    "race_eth_code / race_eth": "RIDRETH3 code / labeled",
    "bmi": "Measured BMI (if present)",
```

```
    "bmi_from_selfreport": "BMI from self-reported height/weight (derived)",
    "weight_change_1y_lb": "Current weight - weight 1y ago (lb)",
    "ever_smoker": "1 if smoked ≥100 cigarettes in lifetime (SMQ020)",
    "current_smoker": "1 if smoke now every/some days (SMQ040)",
}
with open(OUT_DD, "w") as f:
    json.dump(desc, f, indent=2)


print("Saved:", OUT_CSV)
print("Data dictionary:", OUT_DD)
```

```
Saved: data-raw/nhanes2123_prediabetes_features_labeled_clean.csv
Data dictionary: data-raw/nhanes2123_prediabetes_features_datadict.json
```

Data description

Have an initial draft of your data description section. Your data description should be about your analysis-ready data. Data Description

Dataset Overview

Our analysis uses the Continuous NHANES (National Health and Nutrition Examination Survey) dataset collected between August 2021 and August 2023, publicly released by the U.S. Centers for Disease Control and Prevention (CDC) and the National Center for Health Statistics (NCHS). NHANES combines interviews, physical examinations, and laboratory tests to assess the health and nutritional status of adults and children in the United States.

Each observation (row) in our analysis-ready dataset represents one adult respondent (age ≥ 18) who participated in the NHANES 2021–2023 cycle. Each attribute (column) represents either a self-reported questionnaire response, a derived health indicator, or one of the laboratory results used to define the target variable.

Purpose and Funding

NHANES was created to monitor the health and nutrition of the U.S. population and inform public-health policy. The survey is funded and managed by the U.S. Department of Health and Human Services, CDC, and NCHS. Our use of this dataset repurposes these nationally representative data for predictive modeling—to explore whether low-cost survey questions can predict likelihood of prediabetes and guide recommendations for further lab testing.

Observations and Attributes

- Observations: Each row corresponds to a single adult respondent uniquely identified by SEQN.
- Target variable:
  - Prediabetes flag = 1 if respondent's fasting plasma glucose (LBXGLU) is between 100–125 mg/dL or glycohemoglobin (LBXGH) is between 5.7–6.4%.

- Predictor attributes (selected questionnaire variables):
  - Demographics: Age (RIDAGEYR), Sex (RIAGENDR), Race/Ethnicity (RIDRETH3)
  - Weight history: Current and past weight, height, weight-change, BMI, self-reported attempts to lose weight
  - Behavioral indicators: Smoking status, alcohol intake, sleep duration, physical activity (sedentary minutes)
  - Medical history: Prior diagnoses of heart disease, stroke, thyroid disorder, liver/kidney conditions, asthma, hypertension, high cholesterol
  - Mental health: Depression screener items (e.g., poor appetite, feeling down)
  - Derived variables: Self-reported BMI, one-year weight change, ever-smoker (binary), current-smoker (binary)

Data Generation and Collection Process

NHANES uses a stratified multistage probability sample to ensure national representativeness. Participants provide informed consent and complete standardized interviews and physical exams in mobile examination centers. Questionnaire responses are self-reported; laboratory measures (e.g., glucose, HbA1c) are obtained from blood samples.

The inclusion and content of survey modules are designed by NCHS researchers and may change slightly each cycle; these decisions influence which variables are available or missing for given respondents. Nonresponse, refusal, or "Don't know" answers also introduce missingness in self-reported features.

Preprocessing and Cleaning

To produce an analysis-ready dataset, we performed the following steps: 1) Merge: Combined relevant laboratory (GHB, GLU), demographic (DEMO), and questionnaire (e.g., WHQ, SLQ, SMQ, MCQ) .XPT files using SEQN as a join key. 2) Filter: Kept only adult respondents (age $\geq$ 18) with available fasting glucose and/or HbA1c values. Excluded any respondent diagnosed with diabetes or taking insulin/oral medication. 3) Standardize and clean: - Renamed variables for clarity (e.g., PAD680 $\longrightarrow$ sedentary_minutes). - Enforced numeric data types and set implausible values outside realistic human ranges to NaN. - Created derived features (bmi_from_selfreport, weight_change_1y_lb, ever_smoker, current_smoker). - Added labeled versions of categorical codes for readability (e.g., 1 $\longrightarrow$ "Yes," 2 $\longrightarrow$ "No"). 4) Target construction: Labeled each respondent as prediabetic = 1 if LBXGLU $\in$ [100, 125] or LBXGH $\in$ [5.7, 6.4]; otherwise 0. 5) Finalize: Dropped unused columns (e.g., lab variables after target creation, pregnancy items) and exported a cleaned CSV for modeling.

After filtering, the final dataset includes one row per adult respondent and a set of features derived from self-reported survey data, ready for supervised binary classification.

Awareness and Consent

All NHANES participants provide written informed consent prior to participation. They are informed that their data will be used for research and statistical analysis to improve public health.

Data are de-identified before public release, and our analysis uses only the publicly available, anonymized files.

Summary

The resulting dataset represents a nationally representative adult sample with validated laboratory outcomes and extensive self-reported health and behavior variables. It provides an ideal foundation for testing whether self-reported survey data alone can identify individuals who are likely prediabetic and should be prioritized for confirmatory blood testing.

Data limitations

Identify any potential problems with your dataset.

Exploratory data analysis

Perform an (initial) exploratory data analysis.

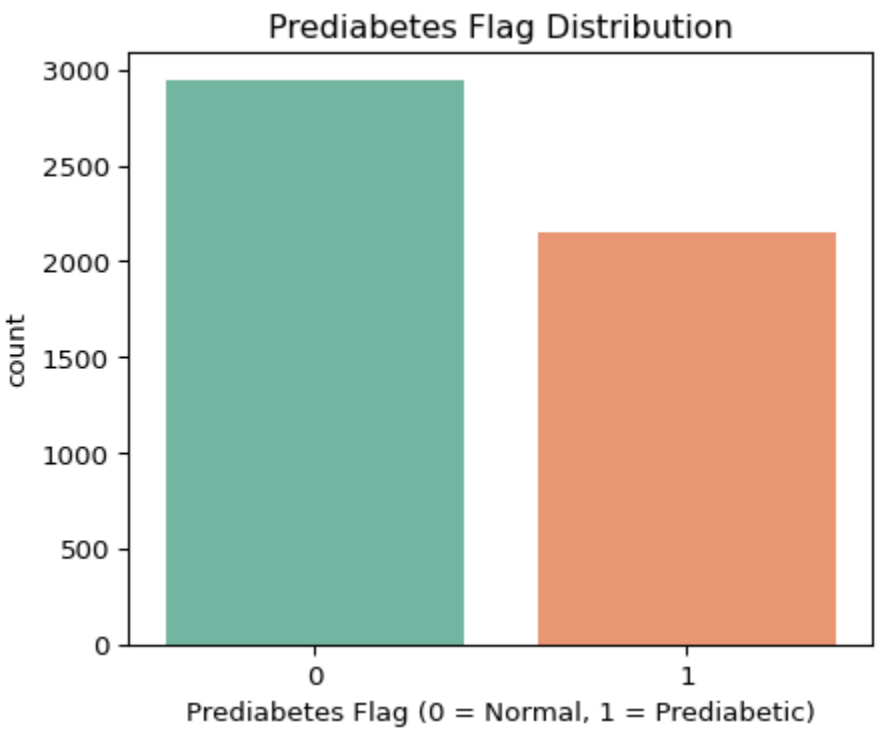
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# load cleaned dataset
df = pd.read_csv("data-raw/nhanes2123_prediabetes_features_labeled_clean.csv")

# basic overview
print(df.shape)
df.head()

# summary of target variable
print(df["prediabetes_flag"].value_counts())
print(df["prediabetes_flag"].value_counts(normalize=True))

plt.figure(figsize=(5, 4))
sns.countplot(x="prediabetes_flag", data=df, palette="Set2")
plt.title("Prediabetes Flag Distribution")
plt.xlabel("Prediabetes Flag (0 = Normal, 1 = Prediabetic)")
plt.show()

# check missing values
missing = df.isna().mean().sort_values(ascending=False)
print(missing.head(10))

# distribution of continuous vars
num_vars = [
    "age_years",
    "bmi_from_selfreport",
    "sleep_hours_weekdays",
```

```
    "sedentary_minutes",
    "drinks_per_day_12mo",
]

for col in num_vars:
    plt.figure(figsize=(6, 4))
    sns.histplot(df[col], kde=True, color="steelblue", bins=30)
    plt.title(f"{col} distribution")
    plt.tight_layout()
    plt.show()

# distribution of some categorical vars
cat_vars = ["sex", "race_eth", "ever_smoker", "current_smoker"]

for col in cat_vars:
    plt.figure(figsize=(6, 4))
    sns.countplot(x=col, data=df, palette="pastel")
    plt.title(f"{col} frequencies")
    plt.xticks(rotation=25)
    plt.tight_layout()
    plt.show()

# relationship between BMI and prediabetes flag
plt.figure(figsize=(6, 4))
sns.boxplot(data=df, x="prediabetes_flag", y="bmi_from_selfreport",
palette="coolwarm")
plt.title("BMI vs Prediabetes Status")
plt.xlabel("Prediabetes Flag (0=No, 1=Yes)")
plt.show()

# relationship between age and prediabetes flag
plt.figure(figsize=(6, 4))
sns.boxplot(data=df, x="prediabetes_flag", y="age_years", palette="mako")
plt.title("Age vs Prediabetes Status")
plt.xlabel("Prediabetes Flag (0=No, 1=Yes)")
plt.show()

# smoking and prediabetes
plt.figure(figsize=(6, 4))
sns.barplot(data=df, x="ever_smoker", y="prediabetes_flag", palette="Set2")
plt.title("Prediabetes Rate by Smoking History")
plt.xlabel("Ever Smoked (1=Yes, 0=No)")
plt.ylabel("Mean Prediabetes Flag")
plt.show()
```

```
(5099, 58)
prediabetes_flag
```

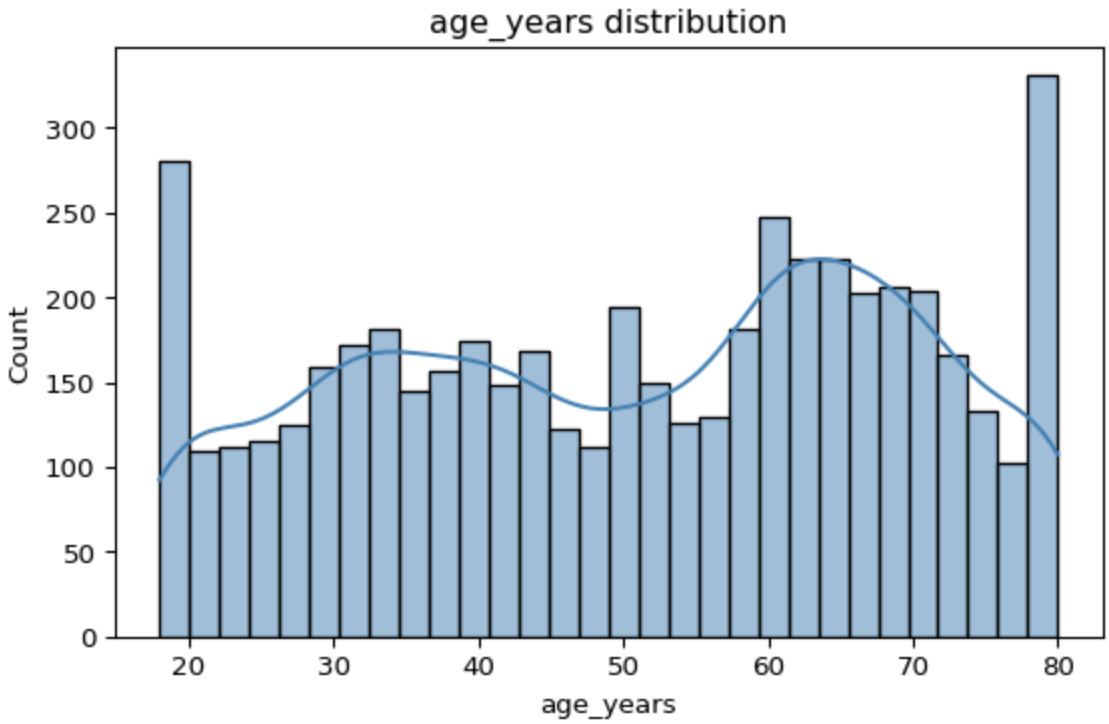
```
0    2944
1    2155
Name: count, dtype: int64
prediabetes_flag
0    0.577368
1    0.422632
Name: proportion, dtype: float64
```

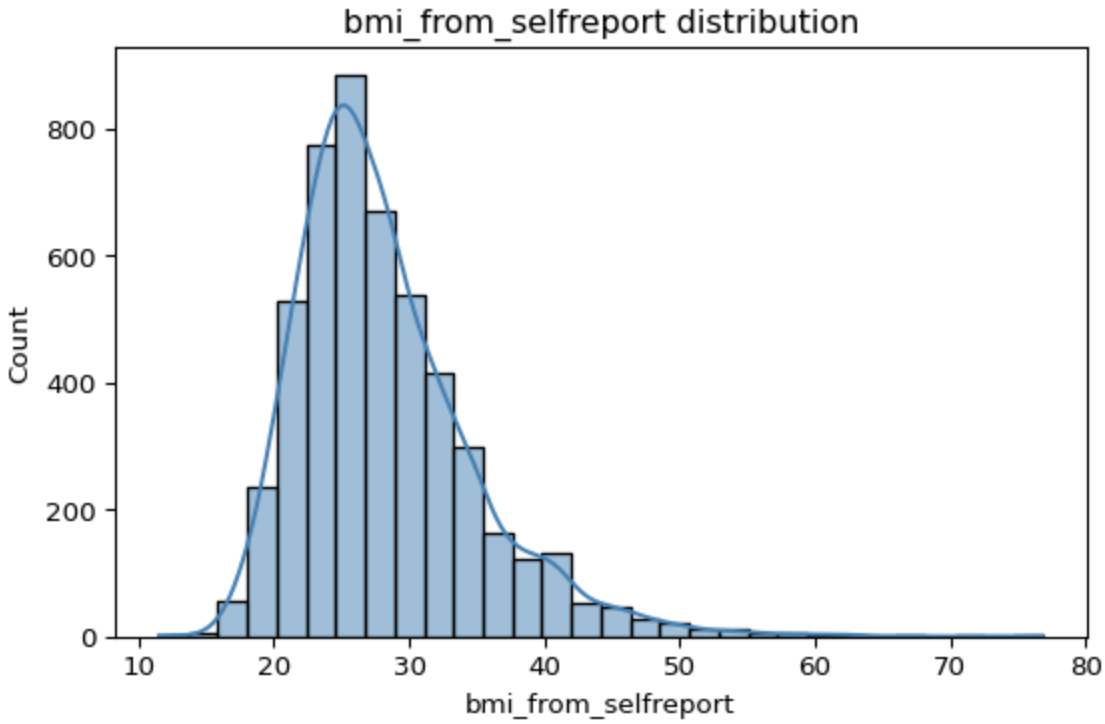
```
/tmp/ipykernel_2652967/3361110983.py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x="prediabetes_flag", data=df, palette="Set2")
```

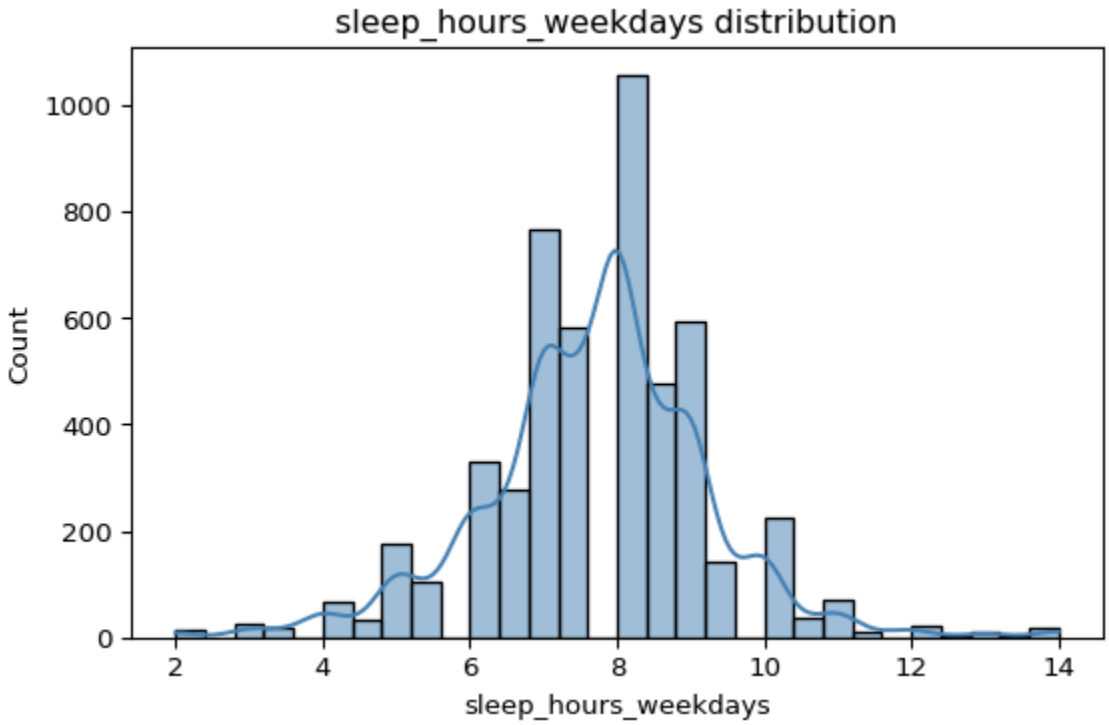


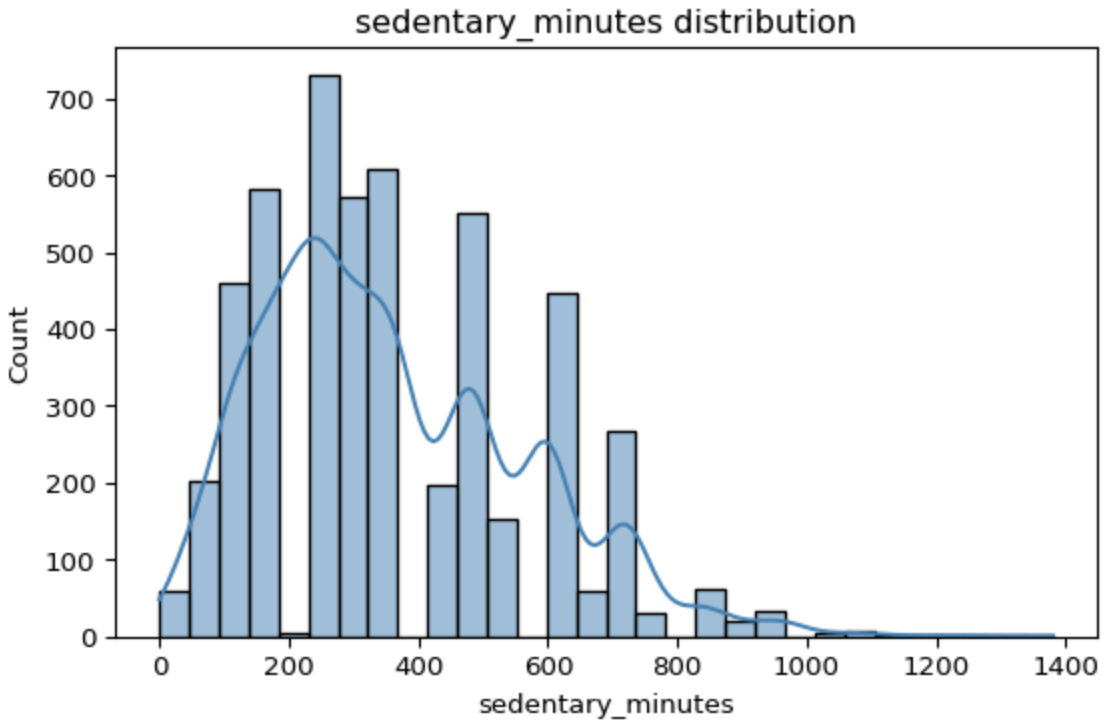

```
liver_autoimmune_hep_ans     1.000000
liver_viral_hep_ans          1.000000
liver_cirrhosis_ans          1.000000
liver_fibrosis_ans           0.999216
liver_fibrosis_code          0.999216
liver_autoimmune_hep_code    0.998039
```

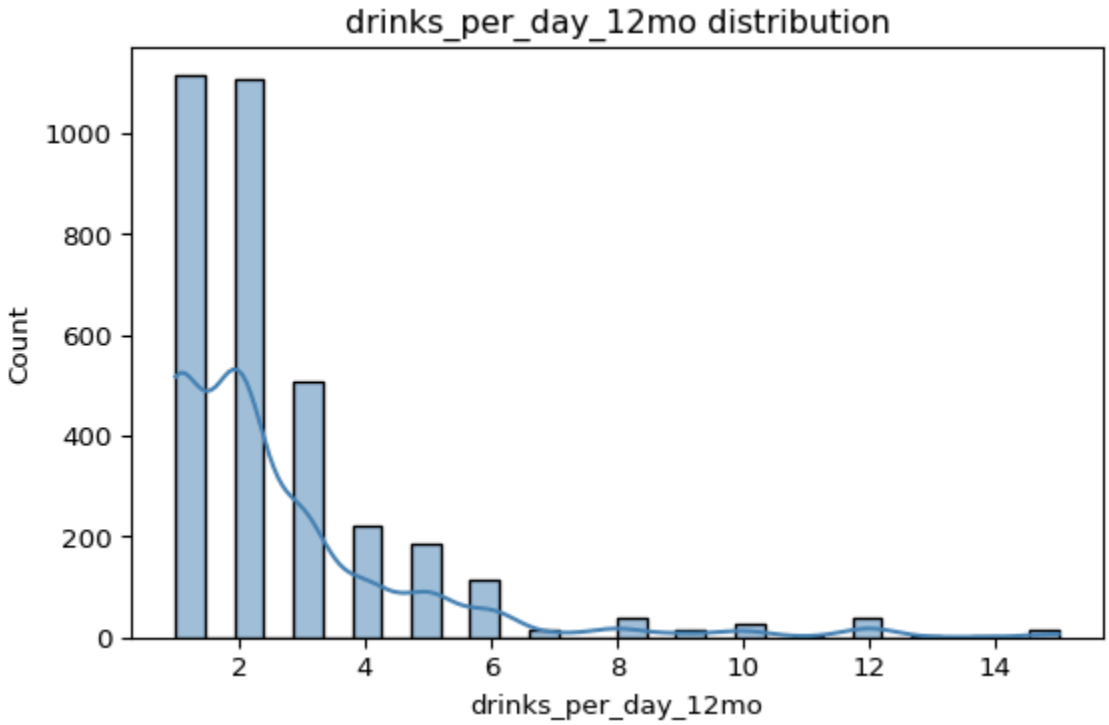
```
liver_cirrhosis_code         0.996274
liver_viral_hep_code         0.992548
liver_fatty_ans              0.977839
liver_fatty_code             0.975289
dtype: float64
```




bmi_from_selfreport distribution
Count
0
200
400
600
800
10
20
30
40
50
60
70
80
bmi_from_selfreport

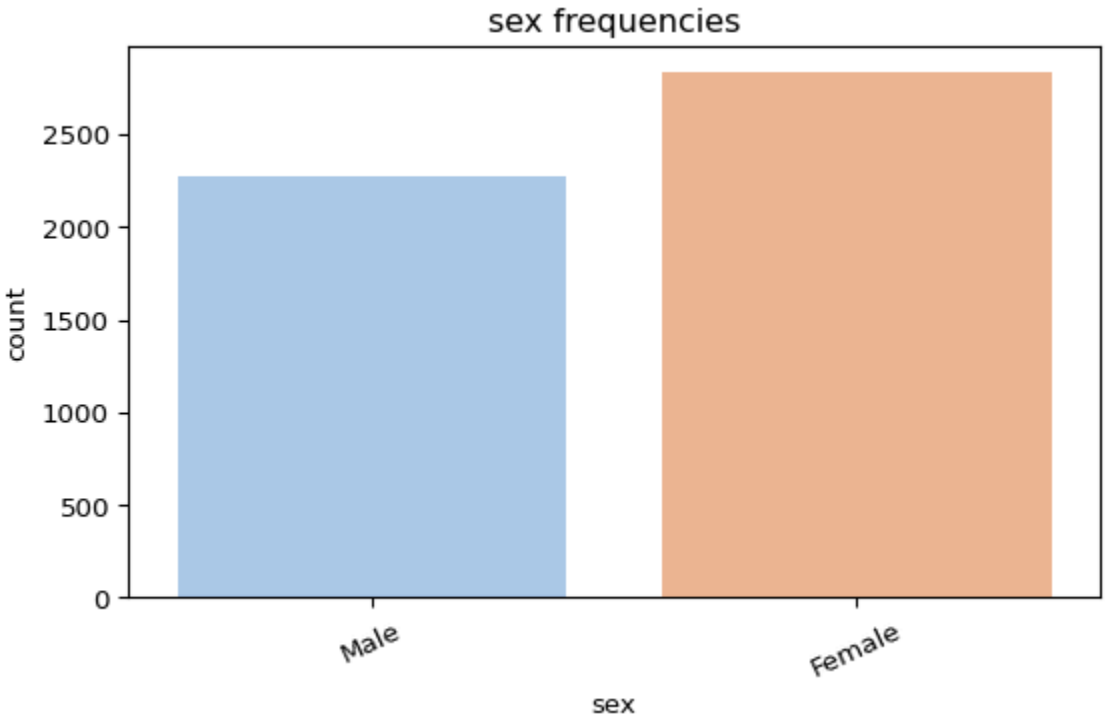

sleep_hours_weekdays distribution
Count
0
200
400
600
800
1000
2
4
6
8
10
12
14
sleep_hours_weekdays



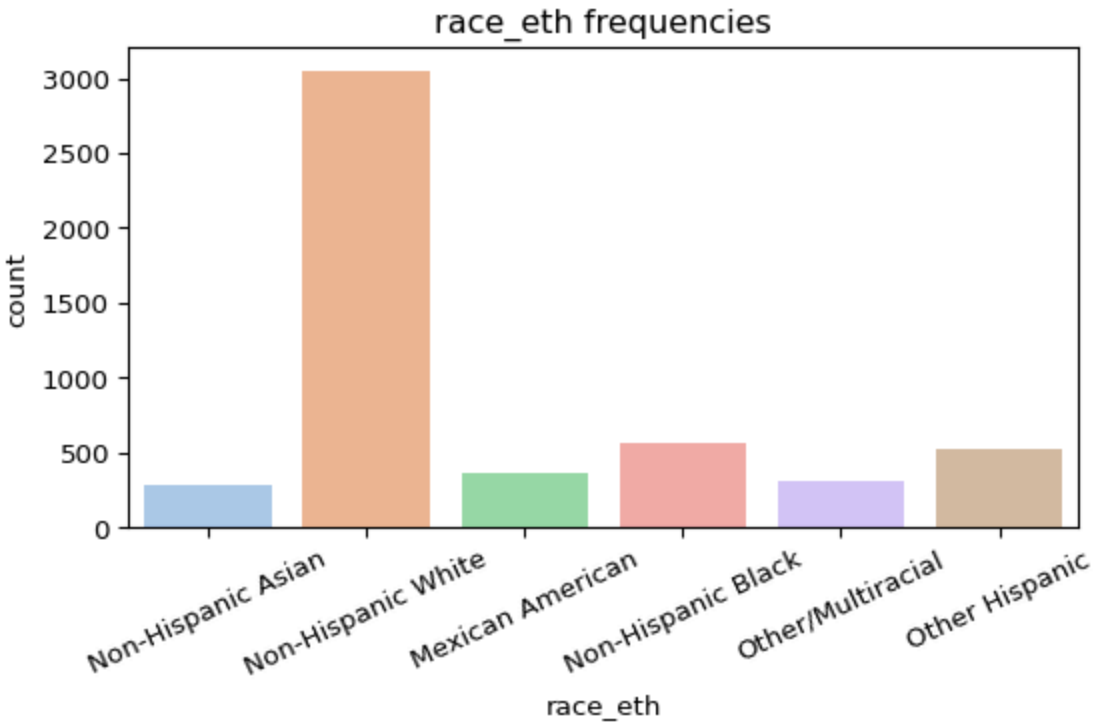



```
/tmp/ipykernel_2652967/3361110983.py:48: FutureWarning:
```

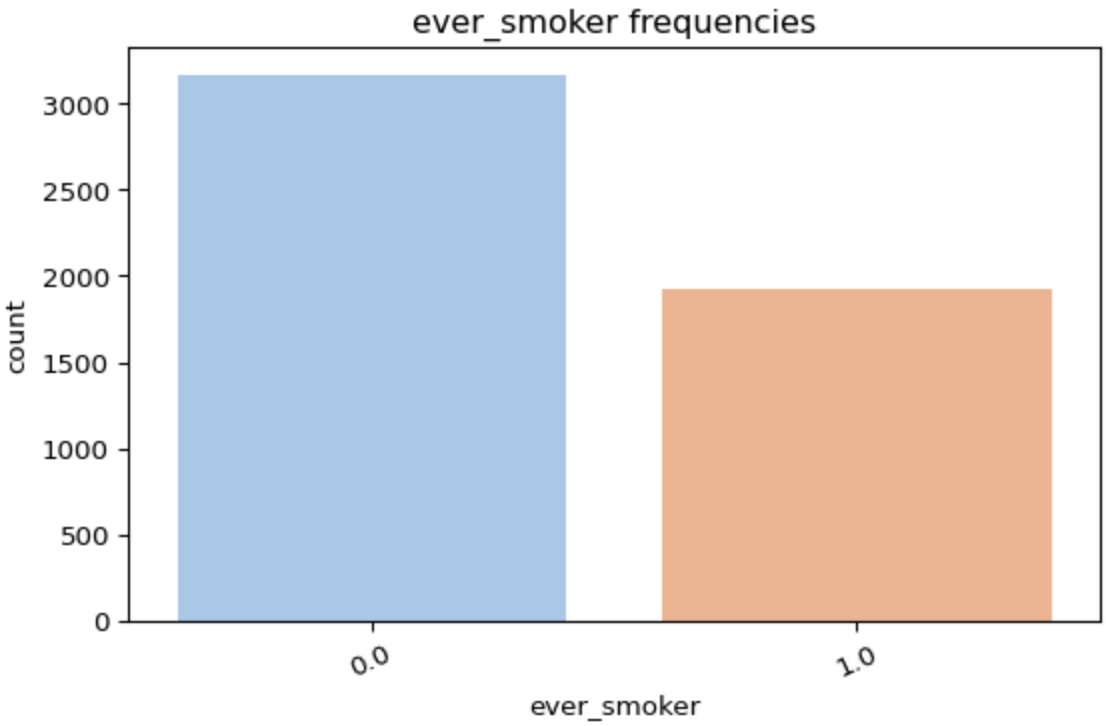
```
Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x=col, data=df, palette="pastel")
```



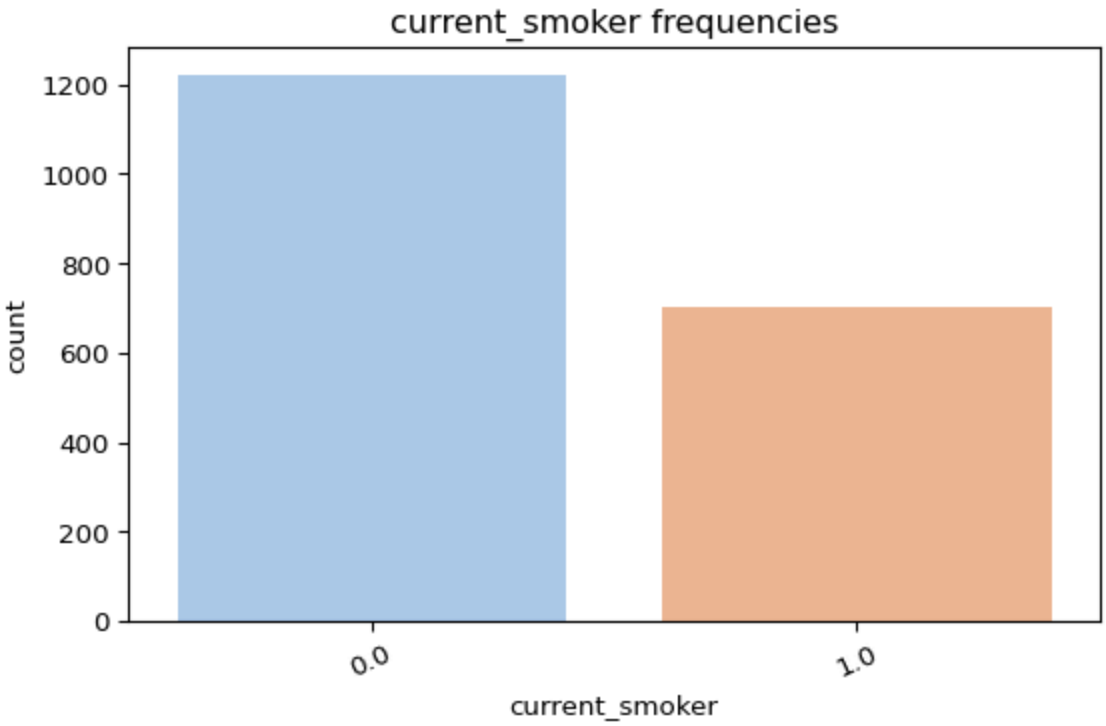

```
/tmp/ipykernel_2652967/3361110983.py:48: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x=col, data=df, palette="pastel")
```



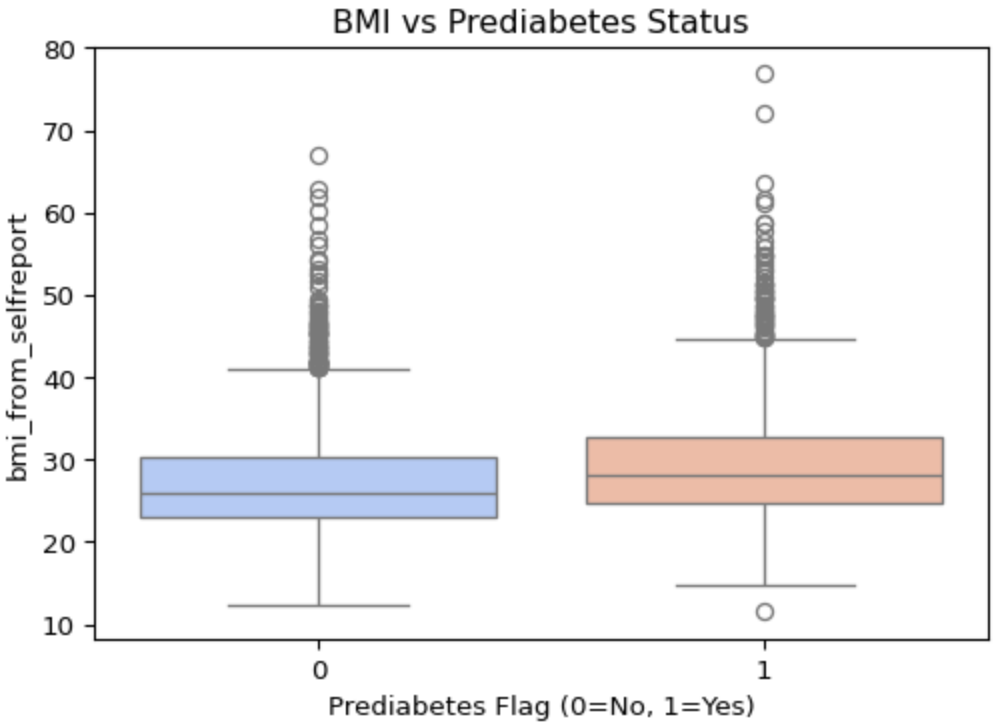

```
/tmp/ipykernel_2652967/3361110983.py:48: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x=col, data=df, palette="pastel")
```



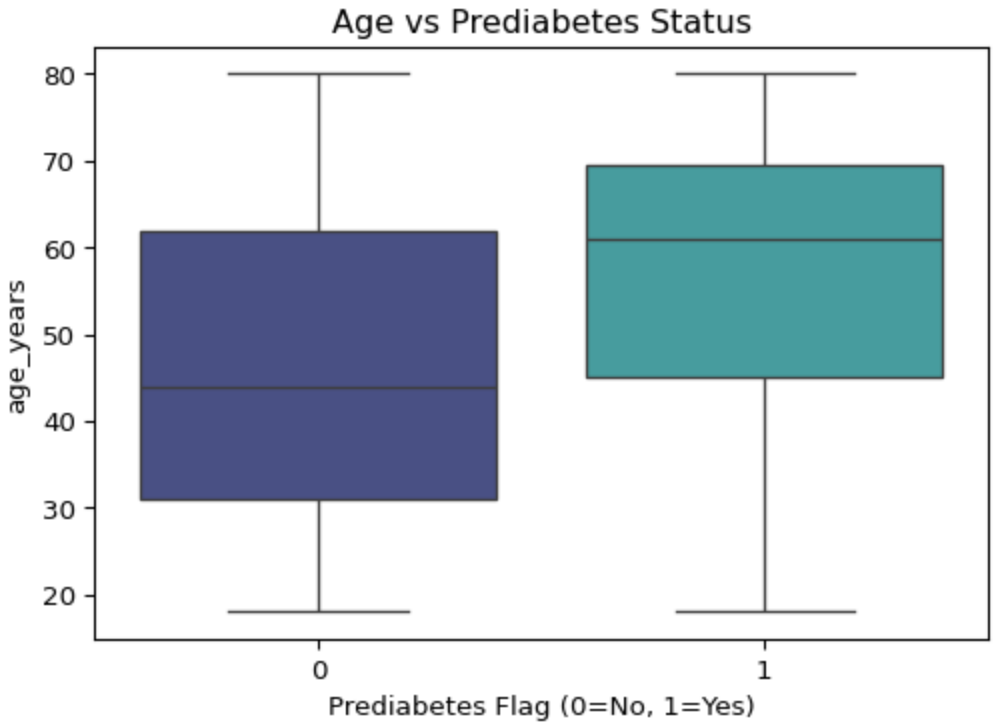

```
/tmp/ipykernel_2652967/3361110983.py:48: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.countplot(x=col, data=df, palette="pastel")
```



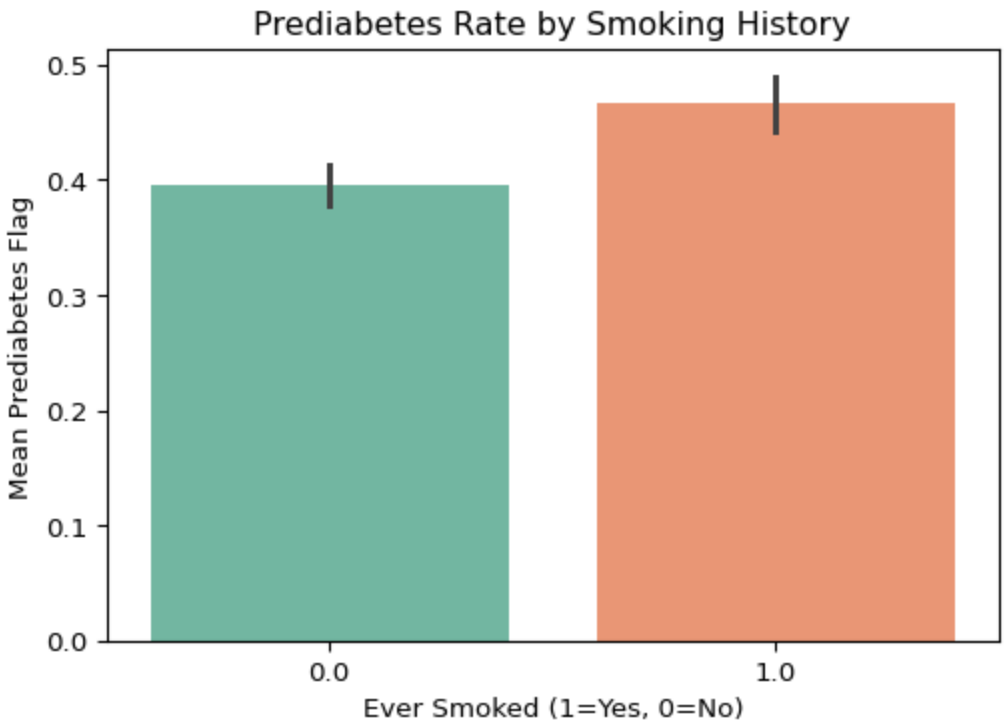

```
/tmp/ipykernel_2652967/3361110983.py:56: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.boxplot(data=df, x="prediabetes_flag", y="bmi_from_selfreport",
palette="coolwarm")
```




```
/tmp/ipykernel_2652967/3361110983.py:63: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.boxplot(data=df, x="prediabetes_flag", y="age_years", palette="mako")
```




```
/tmp/ipykernel_2652967/3361110983.py:70: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.barplot(data=df, x="ever_smoker", y="prediabetes_flag", palette="Set2")
```



Questions for reviewers

List specific questions for your peer reviewers and project mentor to answer in giving you feedback on this phase.